

Örnek çalışma soruları

1. Multithreading ile Asynchronous programlama farkını C# üzerinden açıklayınız.

Multithreading : Paralel kod yürütmesi (CPU-bound işlemler)

1. Multithreading Örneği

csharp

Kopyala

```
using System;
using System.Threading;

class Program {
    static void Main() {
        // Yeni bir thread oluşturup, lambda ifadesi ile "Yeni Thread" yazdırıyoruz
        new Thread(() => Console.WriteLine("Yeni Thread")).Start();

        // Ana thread ise hemen "Main Thread" yazdırır.
        Console.WriteLine("Main Thread");
    }
}
```

Asynchronous : Bloklanmadan beklemek (I/O-bound işlemler)

Örnek 1: Task.Delay ile Gecikme

csharp

Kopyala

```
using System;
using System.Threading.Tasks;

class Program {
    static async Task Main() {
        Console.WriteLine("İşlem başlatılıyor...");
        // 2 saniyelik bekleme: await sayesinde bu süre boyunca Main thread bloke olmaz.
        await Task.Delay(2000);
        Console.WriteLine("İşlem tamamlandı!");
    }
}
```

2. Bir form üzerinde iki adet textBox (textBox1 ve textBox2) ve button1 varsayın. Bu iki giriş alanına yazılan rakamlara göre dinamik olarak listBox ve comboBox Ekleyen button_1 click fonksiyonunu yazınız.

csharp

Kopyala

```
// button1'in Click event handler'ı
private void button1_Click(object sender, EventArgs e)
{
    // textBox1'den ListBox'a eklenecek öğe sayısını alıyoruz
    int listCount = int.Parse(textBox1.Text); // Örn: "5" girilmişse, 5 öğe ekleyecek
    // textBox2'den ComboBox'a eklenecek öğe sayısını alıyoruz
    int comboCount = int.Parse(textBox2.Text); // Örn: "3" girilmişse, 3 öğe ekleyecek

    // Dinamik bir ListBox oluşturuyoruz
    ListBox dynamicListBox = new ListBox(); // Yeni ListBox oluştur
    dynamicListBox.Location = new System.Drawing.Point(20, 150); // Konum ayarla
    dynamicListBox.Size = new System.Drawing.Size(120, 100); // Boyut ayarla
    for (int i = 1; i <= listCount; i++) // 1'den listCount'a kadar döngü
        dynamicListBox.Items.Add(i); // Her adımda öğe ekle

    // Dinamik bir ComboBox oluşturuyoruz
    ComboBox dynamicComboBox = new ComboBox(); // Yeni ComboBox oluştur
    dynamicComboBox.Location = new System.Drawing.Point(150, 150); // Konum ayarla
    dynamicComboBox.Size = new System.Drawing.Size(120, 25); // Boyut ayarla
    for (int i = 1; i <= comboCount; i++) // 1'den comboCount'a kadar döngü
        dynamicComboBox.Items.Add(i); // Her adımda öğe ekle

    // Oluşturulan kontrolleri forma ekliyoruz
    this.Controls.Add(dynamicListBox); // Form'a ListBox ekle
    this.Controls.Add(dynamicComboBox); // Form'a ComboBox ekle
}
```

A	B	C
adres	id	tür
ankara	45435	bia
ankara	34324	is yeri
izmir	23432	bia
izmir	9899	is yeri
istanbul	125	bia
istanbul	65656	is yeri

3. Yukarıdaki alanlara sahip deneme.csv dosyasındaki (id>500) olan verileri listBox1 e ekleyen kod satırlarını yazınız. Bunun için neden bir sınıf dosyasına ihtiyacımız var?

csharp

Kopyala

```
// CSV verilerini nesne olarak tutmak için oluşturulan sınıf
public class Kayit
{
    public string Adres { get; set; } // CSV: adres sütunu
    public int Id { get; set; } // CSV: id sütunu
    public string Tur { get; set; } // CSV: tür sütunu
}
```

csharp

Kopyala

```
// button1'in Click event handler'ı
private void button1_Click(object sender, EventArgs e)
{
    // "deneme.csv" dosyasını StreamReader ile aç
    using (StreamReader sr = new StreamReader("deneme.csv"))
    {
        sr.ReadLine(); // İlk satır başlık satırındır, atla

        string line;
        // Dosya sonuna kadar satır satır oku
        while ((line = sr.ReadLine()) != null)
        {
            // Satırı virgüllerine göre böl (0: adres, 1: id, 2: tür)
            string[] parts = line.Split(',');

            // Kayıt nesnesi oluştur (hata kontrolü yapılmaz)
            Kayit k = new Kayit {
                Adres = parts[0],
                Id = int.Parse(parts[1]),
                Tur = parts[2]
            };

            // Eğer id > 500 ise, kaydı listBox1'e ekle
            if (k.Id > 500)
                listBox1.Items.Add($"{k.Adres} - {k.Id} - {k.Tur}");
        }
    }
}
```

4. Bir .txt dosyasındaki ilk 10 satırı alıp comboBox1'e ekleyen kod satırları için button1_click yazınız.

csharp

Kopyala

```
private void button1_Click(object sender, EventArgs e)
{
    using (StreamReader sr = new StreamReader("ornek.txt"))
    {
        for (int i = 0; i < 10; i++) // İlk 10 satırı oku
        {
            string line = sr.ReadLine(); // Satırı al
            comboBox1.Items.Add(line); // ComboBox'a ekle
        }
    }
}
```

5. dataGridView1 elemanına deneme.csv içindeki verileri tablo olarak bağladığımızı varsayın. Bu tabloyu bindingSource1 üzerinden comboBox1'e bağlayan kodları yazınız.

csharp

Kopyala

```
private void button1_Click(object sender, EventArgs e)
{
    // dataGridView1'e CSV verileri tablo olarak bağlı varsayalım
    // bindingSource1'e dataGridView1'in DataSource'unu ata
    bindingSource1.DataSource = dataGridView1.DataSource;

    // comboBox1'e bindingSource1'i ata
    comboBox1.DataSource = bindingSource1;

    // comboBox'ta hangi sütunun gösterileceğini belirle (örneğin "adres" sütunu)
    comboBox1.DisplayMember = "adres";
}
```

6. LongProcess();

ShortProcess();

Yukarıdaki iki fonksiyonun static async void ve içinde await Task.Delay(1000); ve MessageBox olduğunu varsayarsak ekrana verilecek çıktı sırası nasıl olur?

İki metod da asenkron (static async void) çağrıldığında, hemen ardından await Task.Delay(1000) çağrılır ve her ikisi de paralel olarak 1 saniyelik bekleme durumuna girer. Çağrı sırası LongProcess() sonra ShortProcess() olsa da, asenkron işlemler paralel yürütüldüğü için hangi MessageBox'un önce gösterileceği kesin olarak garanti edilemez. Genelde LongProcess() önce tamamlanırsa onun MessageBox'u önce çıkar ama net bir sıralama yoktur.

7. Aşağıdaki fonksiyonun amacı nedir? Satır satır açıklayınız.

```
public async static void InitializeDatabase()
{
    await ApplicationData.Current.LocalFolder.CreateFileAsync("sqliteSample.db",
CreationCollisionOption.OpenIfExists);
    string dbpath = Path.Combine(ApplicationData.Current.LocalFolder.Path,
"sqliteSample.db");

    using (var db = new SQLiteConnection($"Filename={dbpath}"))
    {
        db.Open();
        string tableCommand = "CREATE TABLE IF NOT " + "EXISTS MyTable
(PPrimary_Key INTEGER PRIMARY KEY, " + "Text_Entry NVARCHAR(2048) NULL)";
        var createTable = new SQLiteCommand(tableCommand, db);
createTable.ExecuteReader();
    }
}
```

```
public async static void InitializeDatabase()
{
    // 1) "sqliteSample.db" adlı bir dosyayı LocalFolder'da açar veya yoksa oluşturur.
    await ApplicationData.Current.LocalFolder.CreateFileAsync(
        "sqliteSample.db",
        CreationCollisionOption.OpenIfExists
    );

    // 2) Uygulamanın LocalFolder yolunu "sqliteSample.db" dosya adıyla birleştirerek
    // tam dosya yolunu (dbpath) elde eder.
    string dbpath = Path.Combine(
        ApplicationData.Current.LocalFolder.Path,
        "sqliteSample.db"
    );

    // 3) "Filename={dbpath}" ifadesiyle belirtilen yolda yeni bir SQLiteConnection oluşturur.
    using (var db = new SQLiteConnection($"Filename={dbpath}"))
    {
        // 4) Veritabanı bağlantısını açar.
        db.Open();

        // 5) "MyTable" adlı bir tablo oluşturmak için gerekli SQL komutunu tanımlar.
        // Tablo, Primary_Key adlı INTEGER PRIMARY KEY ve Text_Entry adlı NVARCHAR(2048) kolonundan oluşur.
        // IF NOT EXISTS ifadesiyle tablo mevcutsa tekrar oluşturulmaz.
        string tableCommand =
            "CREATE TABLE IF NOT EXISTS MyTable " +
            "(Primary_Key INTEGER PRIMARY KEY, " +
            "Text_Entry NVARCHAR(2048) NULL)";

        // 6) SQL komutunu (tableCommand) çalıştırmak için bir SQLiteCommand nesnesi oluşturur.
        var createTable = new SQLiteCommand(tableCommand, db);

        // 7) Komutu yürütür ve tabloyu oluşturur (veya zaten varsa dokunmaz).
        createTable.ExecuteReader();
    }
}
```

Kısacası bu metod, uygulamanın yerel klasöründe bir SQLite veritabanı dosyası oluşturup, içinde **MyTable** adında bir tablo hazırlayarak basit bir veri kaydetme altyapısı kurar.

8. Aşağıdaki kod satırlarını açıklayınız

```
string folderPath = @"C:\Users\maruf\Desktop\images"; // Replace with your folder
path

// Get all image file paths in the folder
string[] imagePaths = GetImagePaths(folderPath);

// Display the paths
int imageCount = imagePaths.Length;
PictureBox[] pictures = new PictureBox[imageCount];
int i = 0;
int x = 500; int y = 50;
while (i < imageCount)
{
    pictures[i] = new PictureBox();
    pictures[i].Location = new Point(x, y);
    pictures[i].SizeMode = PictureBoxSizeMode.StretchImage;
    y = y + 100;
    pictures[i].ImageLocation = imagePaths[i];
    Controls.Add(pictures[i]);
    i++;
}
```

```
// 1) Görüntülerin bulunduğu klasörün yolu belirleniyor.
string folderPath = @"C:\Users\maruf\Desktop\images"; // Kendi klasör yolunuzla değiştirin

// 2) Belirtilen klasördeki tüm resim dosyası yollarını döndüren metod çağırılıyor.
string[] imagePaths = GetImagePaths(folderPath);

// 3) Toplam resim dosyası sayısı hesaplanıyor.
int imageCount = imagePaths.Length;

// 4) Her resim için bir PictureBox oluşturmak amacıyla, PictureBox türünde
// bir dizi (array) oluşturuluyor (dizinin boyutu, resim sayısına eşit).
PictureBox[] pictures = new PictureBox[imageCount];

// 5) Döngü kontrolü için sayaç (index) tanımlanıyor.
int i = 0;

// 6) Resimlerin ekranda konumlandırılacağı başlangıç koordinatları belirleniyor.
int x = 500;
int y = 50;

// 7) Tüm resim dosyaları için PictureBox oluşturma ve ekrana ekleme işlemi
while (i < imageCount)
{
    // 7.1) Yeni bir PictureBox nesnesi oluşturulup dizinin ilgili indeksine atanır.
    pictures[i] = new PictureBox();

    // 7.2) Oluşturulan PictureBox'un ekrandaki konumu (x,y) olarak ayarlanır.
    pictures[i].Location = new Point(x, y);

    // 7.3) PictureBox'un SizeMode özelliği StretchImage olarak ayarlanır,
    // böylece resim, PictureBox boyutuna göre esner.
    pictures[i].SizeMode = PictureBoxSizeMode.StretchImage;

    // 7.4) Y koordinatı 100 piksel arttırılır, böylece sonraki resim
    // dikey olarak alt satıra yerleştirilir.
    y = y + 100;

    // 7.5) PictureBox'un ImageLocation özelliği, resim dosyasının yoluna ayarlanır.
    pictures[i].ImageLocation = imagePaths[i];

    // 7.6) Oluşturulan PictureBox, form üzerindeki kontroller koleksiyonuna eklenir,
    // böylece ekranda görünür hale gelir.
    Controls.Add(pictures[i]);

    i++;
}
```

Kısaca Bu kod, belirli bir klasördeki resim dosyalarının yollarını alıp, her bir resim için PictureBox oluşturup, formda belirlenen konumlarla görüntüler.

9.Aşağıdaki kod parçasını paralel şekle dönüştürünüz

```
DateTime startDateTime = DateTime.Now;
Stopwatch stopWatch = new Stopwatch();
AddToListBox("For Loop Execution start");
```

```
long total;
stopWatch.Start();
for (int i = 0; i < 10; i++)
{
    total = DoSomeIndependentTask();
}

stopWatch.Stop();
```

```
// Başlangıç zamanı ve kronometre ayarlanır
DateTime startDateTime = DateTime.Now;           // Başlangıç zamanı
Stopwatch stopWatch = new Stopwatch();           // Kronometre oluşturulur
AddToListBox("Parallel For Execution start");     // Başlangıç mesajı ekrana yazdırılır
stopWatch.Start();                               // Kronometre başlatılır

// 0'dan 10'a kadar paralel döngü çalıştırılır
Parallel.For(0, 10, i =>
{
    long total = DoSomeIndependentTask(); // Her iterasyonda bağımsız görev çalıştırılır
});

stopWatch.Stop();                               // Kronometre durdurulur
```

9. notifyIcon kullanım amacı nedir?

NotifyIcon, Windows Forms uygulamalarında görev çubuğu sistem tepsisinde (system tray) bir simge göstermeyi sağlar. Bu simge üzerinden kullanıcıya bildirim (ballon tip) sunmak, uygulamayı hızlı erişime açmak veya sağ tıklama menüsü gibi ek işlevler sunmak için kullanılır.

10. İki sınıf oluşturup ortak herhangi değişken değerleri için Linq join eksik kodlarını yazınız.

```
11. List<Person> people = new List<Person>
12. {
13.     new Person { Name = "John", Age = 30, Id=1 },
14.     new Person { Name = "Alice", Age = 25, Id=2 },
15.     new Person { Name = "Bob", Age = 30, Id=3 },
16.     new Person { Name = "Charlie", Age = 25, Id=4},
17.     new Person { Name = "Eve", Age = 35, Id=5 }
18. };
19. List<Person2> people2 = new List<Person2>
20. {
21.     new Person2 { Name = "John", Age = 30, Id=1 },
22.     new Person2 { Name = "Alice", Age = 25, Id=2 },
23.     new Person2 { Name = "Bob", Age = 30, Id=3 },
24.     new Person2 { Name = "Charlie", Age = 25, Id=4},
```

```
25.     new Person2 { Name = "Eve", Age = 35, Id=5 }
26. };
27. var id = 1;
28. var query =
29.     from record in people
```

```

// İki sınıf: Person ve Person2
public class Person
{
    public string Name { get; set; } // Kişinin adı
    public int Age { get; set; }      // Kişinin yaşı
    public int Id { get; set; }       // Ortak eşleşme anahtarı
}

public class Person2
{
    public string Name { get; set; } // Kişinin adı
    public int Age { get; set; }      // Kişinin yaşı
    public int Id { get; set; }       // Ortak eşleşme anahtarı
}

// Listeler
List<Person> people = new List<Person>
{
    new Person { Name = "John", Age = 30, Id = 1 },
    new Person { Name = "Alice", Age = 25, Id = 2 },
    new Person { Name = "Bob", Age = 30, Id = 3 },
    new Person { Name = "Charlie", Age = 25, Id = 4 },
    new Person { Name = "Eve", Age = 35, Id = 5 }
};

List<Person2> people2 = new List<Person2>
{
    new Person2 { Name = "John", Age = 30, Id = 1 },
    new Person2 { Name = "Alice", Age = 25, Id = 2 },
    new Person2 { Name = "Bob", Age = 30, Id = 3 },
    new Person2 { Name = "Charlie", Age = 25, Id = 4 },
    new Person2 { Name = "Eve", Age = 35, Id = 5 }
};

var id = 1; // Örneğin, id'si 1 olan kayıtları seçmek istiyoruz

// LINQ join sorgusu: people ve people2 listelerini ortak Id üzerinden eşleştiriyoruz
var query =
    from record in people
    join record2 in people2 on record.Id equals record2.Id
    where record.Id == id
    select new { record.Name, record.Age, record.Id };

// Sorgu sonucunu kullanmak (örneğin ekrana yazdırmak)
foreach (var item in query)
{
    Console.WriteLine($"{item.Name} {item.Age} {item.Id}");
}

```